



Введение в Frontend

Егор Георгиев

Содержание лекции

1. От HTML+JS к современному фронтенду и React
2. Основы React
3. Инструменты сборки
4. Заключение

От HTML+JS к современному фронтенду и React

Что уже умеем?

- HTML — разметка страницы
- CSS — внешний вид, цвета, шрифты и расположение
- JavaScript — логика, обработка событий и изменение DOM
- DOM — «дерево» элементов, которое браузер строит из HTML

Как развивался фронтенд

Статические страницы: каждый переход — полный перезагруз

- Эра jQuery и «улучшений» интерфейса
- Всё больше логики переезжает в браузер
- Приложение живёт в браузере, а сервер просто отдает данные



```
<!DOCTYPE html>
<html lang="ru">
  <head>
    <meta charset="UTF-8" />
    <title>Привет, DOM</title>
  </head>
  <body>
    <h1 id="title">Привет, мир!</h1>
    <button id="btn">Изменить заголовок</button>

    <script>
      const title = document.getElementById("title");
      const button = document.getElementById("btn");

      button.addEventListener("click", () => {
        title.textContent = "Clicked!";
      });
    </script>
  </body>
</html>
```

```
<!DOCTYPE html>
<html lang="ru">
  <head>
    <meta charset="UTF-8" />
    <title>Привет, DOM</title>
    <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
  </head>
  <body>
    <h1 id="title">Привет, мир!</h1>
    <button id="btn">Изменить заголовок</button>

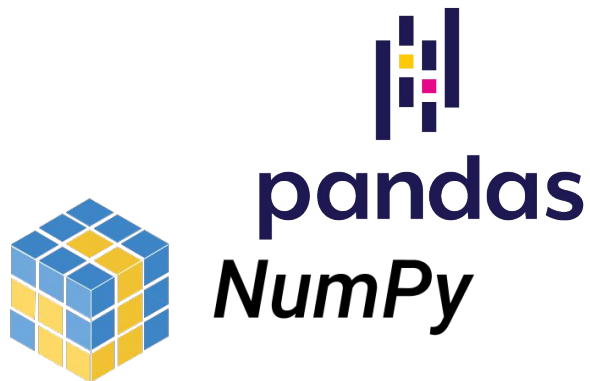
    <script>
      $("#btn").on("click", function () {
        $("#title").text("Clicked!");
      });
    </script>
  </body>
</html>
```

Почему чистого JS становится мало?

- Ручная работа с DOM: `document.querySelector`, `innerHTML`
- Легко запутаться в обработчиках событий и состояниях
- Много дублирования кода для похожих элементов
- Трудно переиспользовать куски интерфейса
- Трудно поддерживать большие команды и большие проекты

Разница между библиотеками и фреймворками

- Библиотека — это набор инструментов
- Фреймворк — это набор библиотек



И как сейчас?

Мир дошел до того, что большая часть логики живет в браузере

Раньше

- Сервер отдает готовый HTML под каждую страницу
- Каждое действие — переход на новую страницу

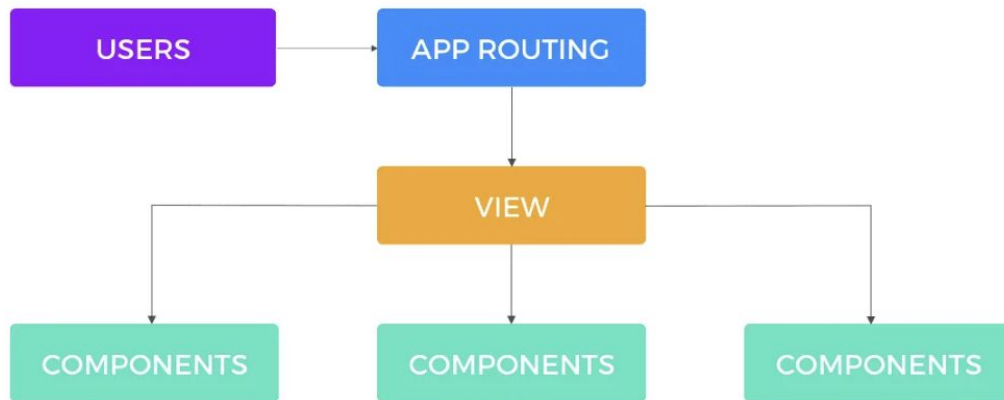
Сейчас:

- Сервер отдает только данные в виде JSON
- Все больше действий без перезагрузки страницы

Такой подход называют **SPA — Single Page Application**

Как появился React?

- Крупные интерфейсы с частыми обновлениями данных
- Нужно было управлять сложным состоянием
- Идея: разбить UI на компоненты
- Идея: описать интерфейс декларативно, а не императивно
- Но оставить JavaScript как главный язык



Декларативный подход и Virtual DOM

Как думать: «что показать», а не «как менять DOM»

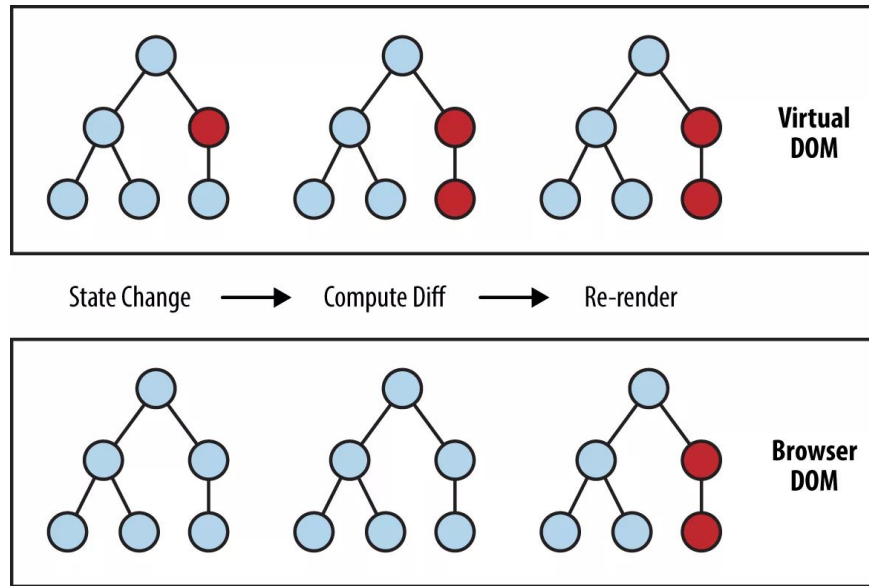
Императивно — мы сами ищем элементы и пошагово меняем DOM (как делать?)

Декларативно — описываем как должен выглядеть интерфейс при таком состоянии (что делать?)

Декларативный подход и Virtual DOM

- React строит свое дерево – Virtual DOM
- Когда состояние меняется, React рисует заново Virtual DOM
- Затем React сравнивает что изменилось и меняет только то, что изменилось

*Virtual DOM – черновик интерфейса
в памяти*



Интерфейс как конструктор из компонентов

- Компонент — это маленький самостоятельный блок интерфейса
- Страница = набор компонентов
- Компоненты можно и нужно переиспользовать

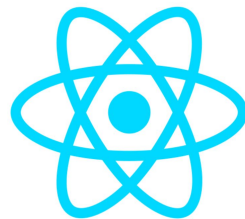
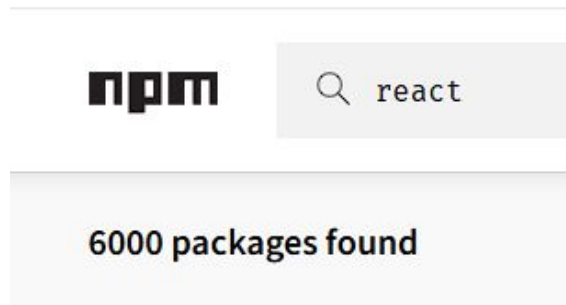
```
<>
  <Header />
  <Navigation />
  <PostList>
    <Post />
    <Post />
    <Post />
  </PostList>
  <Footer />
</>
```

React не один

- Node JS и npm — менеджер пакетов
- Bundler — собирает код в упакованный вариант
- dev-server — сервер для разработки

React — это только библиотека

<https://react.dev/>



Основы React

Что такое компонент в React?

Компонент — это функция, которая возвращает кусок интерфейса

- На вход компонент получает **props**
- На выходе — описание интерфейса: какие теги, с каким текстом, классами и так далее

```
function HelloUser(  
  {  
    name,  
  }  
) {  
  return <h1>Привет, {name}!</h1>;  
}
```

Базовый синтаксис

- Компонент в React — это обычная JS-функция
- Название компонента с большой буквы
- Функция возвращает JSX
- Обязательно возвращать один корневой элемент (или `<>...</>` — фрагмент)

```
function Button({ label }) {  
  return <button>{label}</button>;  
}
```

```
function Title() {  
  return <h1>Мой первый интерфейс на компонентах</h1>;  
}
```

```
function App() {  
  return (  
    <div className="app">  
      <Title />  
      <p>Выберите действие:</p>  
  
      <div className="actions">  
        <Button label="Сохранить" />  
        <Button label="Отмена" />  
      </div>  
    </div>  
  );  
}
```

JSX

```
1 function InnerComponent() {  
2   return <World/>  
3 }  
4  
5 function Component() {  
6   return <h1>Hello <InnerComponent /> {1 === 1}</h1>;  
7 }  
8
```

-JS

```
function InnerComponent() {  
  const $ = _c(1);  
  let t0;  
  if ($[0] === Symbol.for("react.memo_cache_sentinel")) {  
    t0 = <World/>;  
    $[0] = t0;  
  } else {  
    t0 = $[0];  
  }  
  return t0;  
}  
  
function Component() {  
  const $ = _c(1);  
  let t0;  
  if ($[0] === Symbol.for("react.memo_cache_sentinel")) {  
    t0 = (  
      <h1>  
        | Hello <InnerComponent /> {true}  
      </h1>  
    );  
    $[0] = t0;  
  } else {  
    t0 = $[0];  
  }  
  return t0;  
}
```

JSX

Браузер не понимает JSX

```
function HelloUser(props) {  
  return React.createElement(  
    "h1",  
    { className: "title" },  
    "Привет, ",  
    props.name,  
    "!"  
  );  
}
```

ИЛИ

```
function HelloUser({ name }) {  
  return (  
    <h1 className="title">  
      Привет, {name}!  
    </h1>  
  );  
}
```

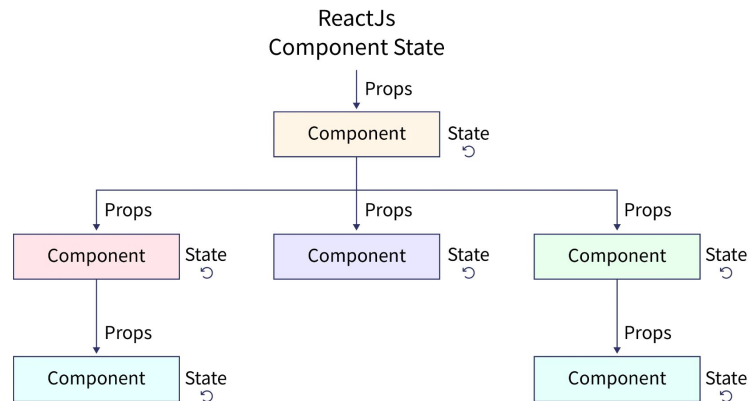
Состояние компонента

У интерфейса есть **данные, которые меняются со временем**:

- количество лайков
- введённый текст в инпуте
- открыт/закрит модальное окно
- текущая вкладка/шаг формы

props — данные, которые приходят «снаружи», компонент их **не меняет**

state — данные, которые живут **внутри** компонента и **могут меняться**



Состояние компонента

state в компонентах хранится через хук `useState`

useState — это функция из React, которую мы вызываем внутри компонента

Возвращает пару значений:

- текущее состояние
- функцию, которая это состояние обновляет

```
import { useState } from "react";

function Counter() {
  const [count, setCount] =
    useState(0);

  function handleClick() {
    setCount(count + 1);
  }

  return (
    <button onClick={handleClick}>
      Нажато {count} раз
    </button>
  );
}
```

Состояние компонента

Как React перерисовывает компонент?

При вызове `setCount(...)`:

- React запоминает новое значение состояния
- заново вызывает функцию компонента
- заново строит JSX на основе нового state

```
import { useState } from "react";

function Counter() {
  const [count, setCount] =
    useState(0);

  function handleClick() {
    setCount(count + 1);
  }

  return (
    <button onClick={handleClick}>
      Нажато {count} раз
    </button>
  );
}
```

Обработка событий

События в React — это **атрибуты на JSX-элементах**:

- `onClick`, `onChange`, `onSubmit`, `onKeyDown` и т.д.

Значение события — **функция**, а не строка:

- правильно: `onClick={handleClick}`
- неправильно: `onClick="handleClick()"` (так было в чистом HTML)

Условный рендеринг

В React мы можем показывать или не показывать куски кода JSX в зависимости от

- state
- props

```
<h1>{isLoggedIn ? "Личный кабинет" : "Гость"}</h1>
```

```
if (isLoading) {  
  return <p>Загрузка...</p>;  
}
```

```
{isLoggedIn && (  
  <p>  
    У вас {hasNotifications ?  
    "есть" : "нет"} новых уведомлений  
  </p>  
)}
```

Списки в React

Частая задача – вывести список элементов

Для этого используется массив `[]` и метод массива `.map(...)`

Обязательный атрибут **key**, но зачем?

- **key** помогает React отличать элементы списка друг от друга
- должен быть уникальным и стабильным для элемента

```
const posts = [  
  { id: 1, title: "Первый пост" },  
  { id: 2, title: "Второй пост" },  
  { id: 3, title: "Третий пост" },  
];
```

```
export default function Posts() {  
  return (  
    <ul>  
      {posts.map((post) => (  
        <li key={post.id}>  
          {post.title}  
        </li>  
      ))}  
    </ul>  
  );  
}
```

Что делать после рендера?

useEffect — хук для **побочных эффектов**:

- запросы к серверу
- подписки на события
- работа с localStorage
- изменение document.title и т.п.

```
useEffect(() => {  
    function handleResize() {  
        setWidth(window.innerWidth);  
    }  
  
    window.addEventListener("resize",  
        handleResize);  
  
    return () => {  
        window.removeEventListener("resize",  
            handleResize);  
    };  
}, [])
```

Что делать после рендера?

- В момент рендера компонента **window**, **document** могут быть недоступны, так как сервер не знает о таком
- **useEffect** никогда не запускается на сервере

```
import { useState, useEffect } from
"react";

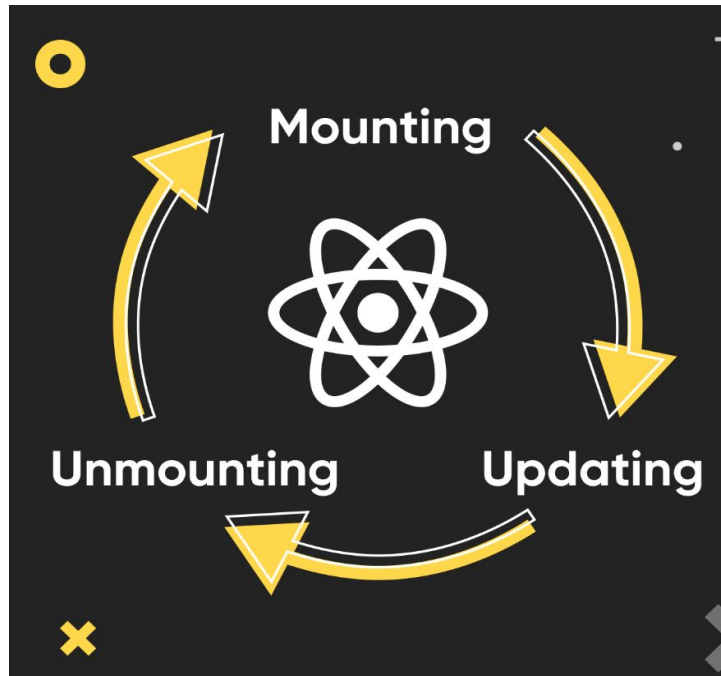
export default function BadWindow() {
  const [width, setWidth] =
    useState(window.innerWidth);

  useEffect(() => {
    console.log("ширина:", width);
  }, [width]);

  return (
    <p>Текущая ширина окна: {width}px</p>
  );
}
```

Жизненный цикл компонента

- Компонент появляется на странице
- Обновляется, когда меняется props или state
- Удаляется, когда больше не нужен



Инструменты сборки

Зачем нужны сборщики?

Сборщик понимает импорты, собирает код в один-два файла, преобразует JSX и TypeScript в обычный JS, и на выходе даёт браузеру аккуратный, понятный ему JavaScript

Браузер не знает **import** и **export**, если не указать `type="module"`

Браузер не поймет, если мы напишем

```
import React from "react";
```

Ему нужен полный путь/URL: `./vendor/react.js` или что-то вроде CDN

А если у нас 100 модулей? Браузер сделает 100 запросов!

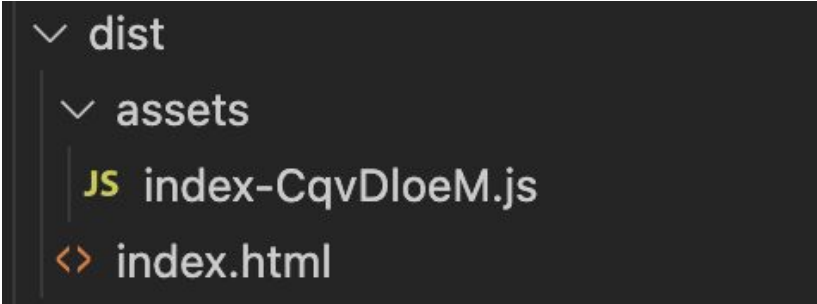
Зачем нужны сборщики?

Сборщик умеет:

- Выкидывать неиспользуемый код
- Минифицировать файлы
- Собирает все в несколько оптимизированных файлов (бандлы)

```
<body>
  <div id="app"></div>

  <script src="utils.js"></script>
  <script src="api.js"></script>
  <script src="app.js"></script>
</body>
```



```
✓ dist
  ✓ assets
    JS index-CqvDloeM.js
    <> index.html
```

Управление зависимостями

npm — менеджер пакетов для JavaScript

- аналог pip в Python
- ставит библиотеки в папку node_modules

package.json — «паспорт» проекта

- хранит название проекта, версию, скрипты
- список зависимостей



webpack

webpack — это сборщик модулей для JavaScript-приложений

- Берет много файлов:
 - App.jsx, Button.jsx, styles.css, картинки, шрифты
- Понимает import / export между ними
- Прогоняет через лоадеры:
 - JSX → обычный JS
 - TypeScript → JS
 - SCSS → CSS
- Складывает всё в несколько файлов

```
// webpack.config.js
module.exports = {
  entry: './src/main.jsx',
  output: {
    filename: 'bundle.js',
    path: __dirname +
    "/dist",
  },
};
```

<https://webpack.js.org/>

webpack

- Медленный старт проекта
- Медленные обновления кода
- Сложная конфигурация
- Высокий порог входа
- webpack был придуман, когда не было нативных ES-модулей в браузере



Vite

- Быстрый старт dev-сервера
- Быстрое обновление кода: при изменении пересобирается только измененный файл
- Меньше конфигурации
- Современный стек

```
import { defineConfig } from 'vite'  
import react from '@vitejs/plugin-react'
```

```
// https://vite.dev/config/  
export default defineConfig({  
  plugins: [react()],  
})
```



Что можно сделать в конфиге?

- **entry / output** — откуда брать код и куда класть сборку
- **alias'ы путей** — чтобы писать `@/components/Button` вместо длинных относительных путей
- **загрузчики/лоадеры** — как обрабатывать CSS, картинки, TypeScript, JSX
- **плагины** — минификация, HTML-шаблон, анализ бандла и т.д.
- **dev-server** — порт, прокси и так далее

<https://vite.dev/config/>

Заключение

Структура проекта

- src/styles/ — стили
- src/components/ — компоненты
- src/pages/ — страницы верхнего уровня
- src/features/ — функциональные модули
- src/lib — вспомогательные функции

```
├─ main.jsx
├─ App.jsx
├─ assets/
│   └─ logo.svg
│       └─ images/
│           └─ example.png
├─ styles/
│   └─ globals.css
│       └─ App.css
├─ components/
│   └─ layout/
│       ├── Header.jsx
│       ├── Footer.jsx
│       └─ Navigation.jsx
│   └─ ui/
│       ├── Button.jsx
│       └─ Card.jsx
│   └─ common/
│       └─ Loader.jsx
├─ pages/
│   ├── HomePage.jsx
│   └─ PostsPage.jsx
├─ features/
│   └─ posts/
│       ├── components/
│       │   ├── PostList.jsx
│       │   └─ PostItem.jsx
│       ├── api/
│       │   └─ postsApi.js
│       └─ hooks/
│           └─ usePosts.js
├─ hooks/
│   └─ useWindowSize.js
└─ lib/
    └─ apiClient.js
```


Работа с API

Для получения данных с сервера чаще всего используется

- встроенный fetch
- или стороннюю библиотеку axios

```
const api = axios.create({  
  baseUrl: "/api",  
  timeout: 10000,  
  headers: {  
    "Content-Type":  
"application/json",  
  },  
});
```

```
/**  
 * Получить список постов  
 * GET /api/posts  
 */  
export async function getPosts() {  
  const response = await  
api.get("/posts");  
  return response.data;  
}
```

Вопросы?